LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Software Testing and Input Validation using Python, pytest and Hypothesis

Student Name	Vilohith Sree Sai Reddy Daggolu
Repository	SE_LAB_1
Lab Folder	Lab 7
Programming Language	Python 3.11.9
Testing Tools	pytest 9.2.0 and Hypothesis 6.170.15

1. Aim

The aim of this lab is to use property-based fuzz testing to identify input-validation bugs in a Python warranty-claims application. Hypothesis generates many inputs, including unexpected and malformed values, so that functions can be checked beyond a small set of manually selected examples.

2. Environment and Tools

Component	Version / Purpose
Python	3.11.9 — implementation language
pytest	9.2.0 — test runner
Hypothesis	6.170.15 — property-based input generation

3. Application Functions Under Test

Function	Input format	Purpose
parse_claim(text)	plan:months	Splits a plan name and warranty age in months; formats the plan and converts mon
age_average(ages_csv)	12,24,36	Converts comma-separated product ages to integers and calculates the integer ave
payable_charge(text)	charge|discount	Applies 18% tax to the charge, subtracts the discount and prevents a negative paya

4. Initial Fuzzing and Bugs Identified

The first Hypothesis tests supplied arbitrary text strings to the three functions. The initial test run collected all three tests and reported three failures. The failures showed that malformed inputs could cause unhandled conversion or splitting errors.

Function	Hypothesis failing input	Observed problem
parse_claim()	':' and ''	The input did not contain the required plan/months fields, or the months field was empty; s
age_average()	''	An empty CSV field was converted to int, raising ValueError.
payable_charge()	''	The required charge and discount fields were not present; unpacking raised ValueError.

5. Corrections and Input Validation

• **parse_claim():** Checks that the input is text, contains exactly one colon, and has non-empty plan and month fields. It converts months to a float and rejects negative or non-finite values.

• **age_average():** Checks that the input is text, rejects empty CSV entries, converts each value to an integer, and rejects negative ages before calculating the average.

• **payable_charge():** Checks that the input is text, contains exactly one pipe, and has both fields populated. It converts the values to floats and rejects negative or non-finite values before applying tax and discount.

• Invalid data is rejected using controlled **ValueError** exceptions rather than allowing accidental conversion, unpacking or division errors.

6. Final Hypothesis Test Suite

The final suite contains six tests. It checks valid generated inputs and verifies that selected malformed inputs are rejected. The valid tests check the returned values, while the invalid-input tests expect ValueError to be raised.

Test	What it verifies	Result
test_parse_claim_valid	Correct plan formatting and month conversion.	**PASSED**
test_parse_claim_invalid_format	Malformed claim formats are rejected.	**PASSED**
test_age_average_valid	Correct integer average for generated age lists.	**PASSED**
test_age_average_invalid	Empty and malformed CSV values are rejected.	**PASSED**
test_payable_charge_valid	Correct tax/discount calculation.	**PASSED**
test_payable_charge_invalid	Malformed charge/discount inputs are rejected.	**PASSED**

FINAL RESULT

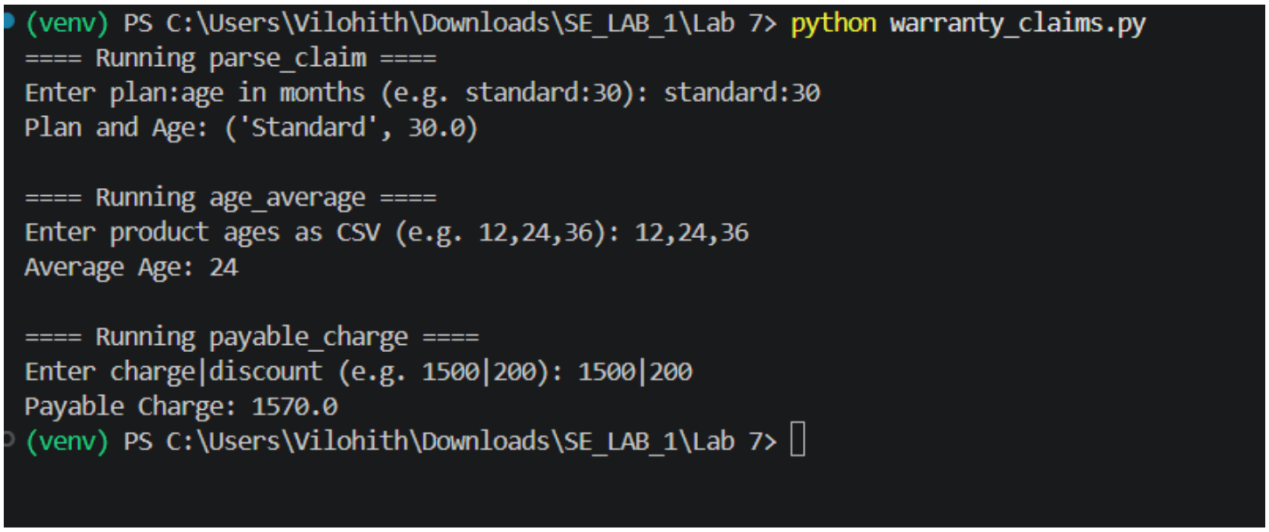
6 / 6 TESTS PASSED

Execution time: 0.92 seconds

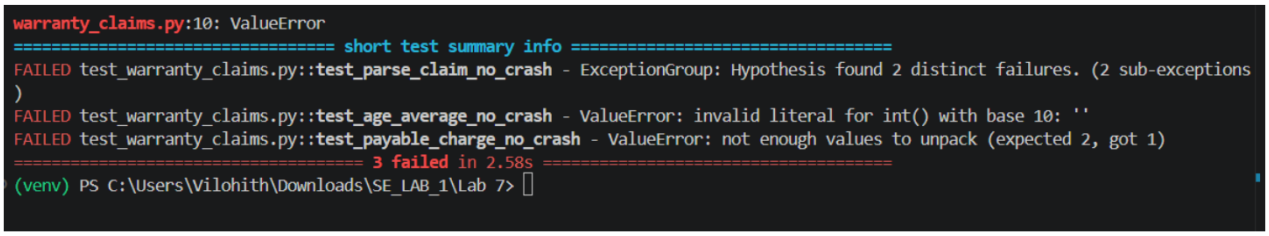
7. Screenshot Evidence

The following evidence images were taken from the one-page PDF supplied with the lab work. The four terminal captures are presented separately for readability.

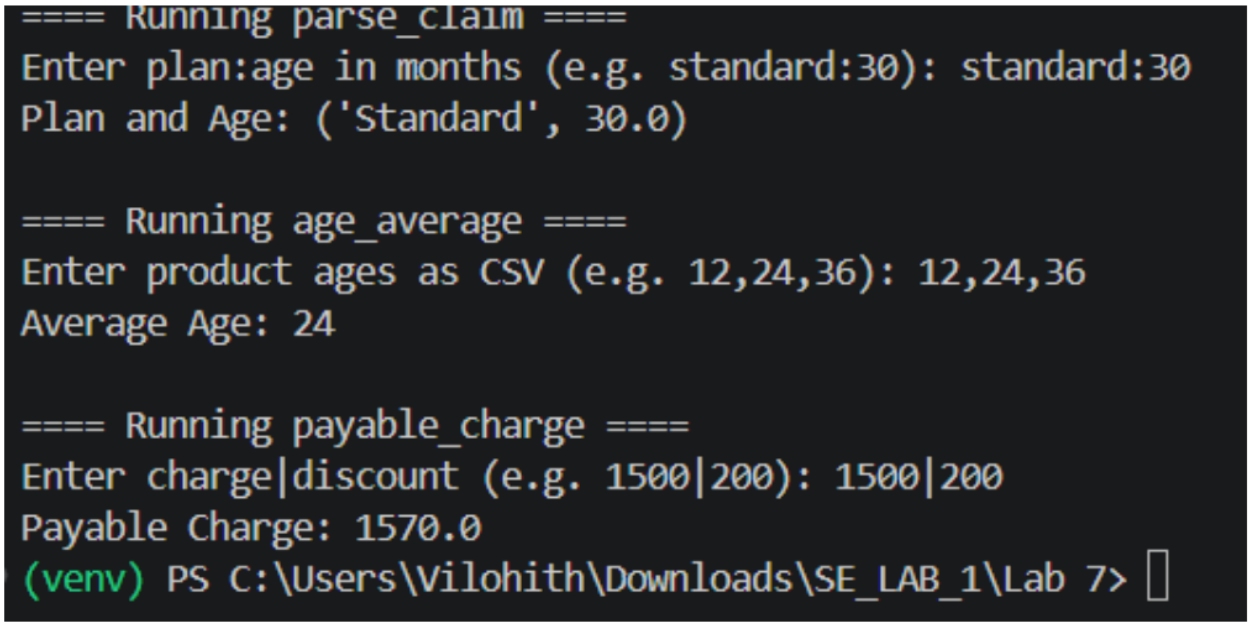
Screenshot 1. Original application run with valid sample inputs and the expected outputs.



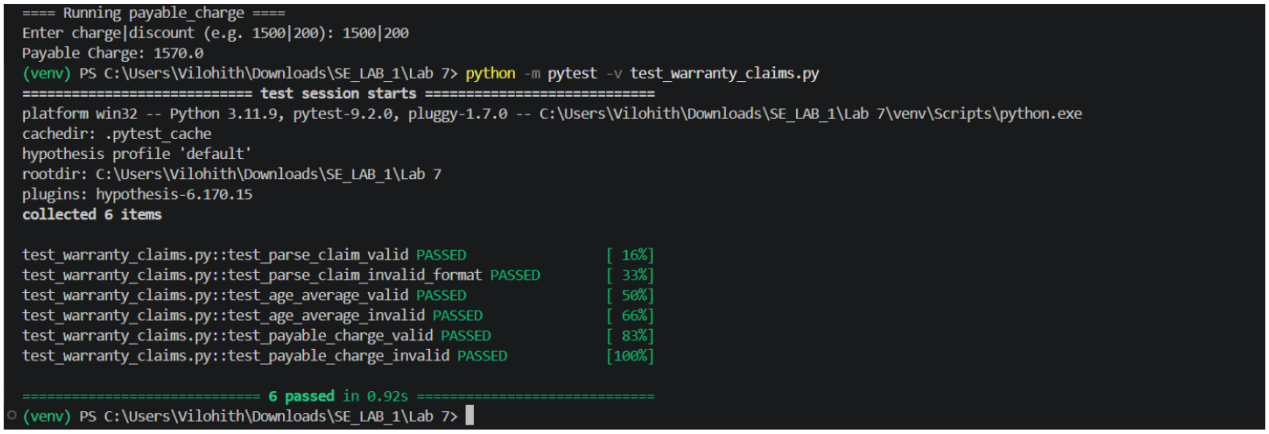
Screenshot 2. Initial fuzzing run: Hypothesis exposed failures in the three input-processing functions.



Screenshot 3. Corrected application run showing the expected outputs for all three functions.



Screenshot 4. Final pytest run showing all six Hypothesis tests passing.



8. Conclusion

The lab demonstrated how property-based fuzz testing can expose input-validation defects that may not be found by testing only normal inputs. Hypothesis identified malformed inputs that caused failures in parse_claim(), age_average() and payable_charge(). The application was updated with input-format, conversion and range checks. The final test suite then passed all six tests, covering both valid behavior and rejection of selected invalid inputs.

Submitted by: Vilohith Sree Sai Reddy Daggolu